IA Aplicada com LLMs

Aula 14 — Cadeia, imagem, rede, e o fecho

Roteiro

1. **O caso de encerramento** — OpenClaw, e por que ele não foi defeito de arquitetura
2. **A imagem** — o `Dockerfile` lido como superfície
3. **MCP e *skills*** — a descrição é entrada; e o pacote que o modelo inventa
4. **Com que credencial o agente age** — o deputado confuso
5. **Red team como suíte** — utilidade e resistência
6. **Registrar, recuperar, aceitar** — e o fecho

Código: `codigo/04.1` a `codigo/04.3`.

Onde os sistemas caem

Enquanto se discute qual padrão de projeto resiste a injeção, sistemas caem por

```
escutar em todas as interfaces com autenticação desligada
instalar extensões de um repositório sem revisão
```

Parte 1

O caso de encerramento

Aberto em casos de uso, retomado em memória, fechado aqui

OpenClaw, *runtime* de agente auto-hospedado e de código aberto: adoção explosiva, e o primeiro incidente de escala da área no início de 2026:

```
dezenas de milhares de instâncias EXPOSTAS na internet pública,
em dezenas de países

mais de 90% delas com falha de autenticação (num dos levantamentos)

milhares exploráveis por execução remota de código

varredura por atacantes iniciada NO MESMO DIA do anúncio público

centenas de extensões maliciosas no repositório de extensões
— mais de um décimo do catálogo
```

Um número que cresce entre duas aulas

As aulas de **casos de uso e de memória** citam *"mais de trinta mil instâncias"*.

A cifra corrente é de **ordem maior**, e os levantamentos divergem entre quarenta e cento e trinta mil, conforme o método de varredura.

> A diferença não é erro de uma fonte: o número cresceu **entre duas aulas do mesmo semestre**, e esse crescimento é um dado sobre a velocidade com que uma
> superfície exposta é encontrada.

A leitura, e ela determina a ordem do curso

Camada	Defeito	De quem é
rede	escutava em `0.0.0.0` em vez de `127.0.0.1`	**deploy (produção)**
autenticação	desligada por omissão	padrão do produto
dependência	registro de extensões sem curadoria	**supply chain (MCP)**
execução	extensões com acesso ao sistema de arquivos	sandbox
arquitetura de agente	—	**SEM DEFEITO**

E é isso que justifica segurança vir DEPOIS de produção

> Ensinar defesa arquitetural **antes** de existir uma superfície publicada seria defender o flanco errado.
>
> E descrever a superfície **antes que ela exista** seria descrever o vazio.

Há um segundo efeito, mais incômodo e mais útil:

um sistema pode ter arquitetura impecável e cair por um padrão de configuração. O deck anterior **não é suficiente**, e dizê-lo é honesto.

Parte 2

A imagem, relida como superfície

O `Dockerfile`, anotado como superfície

```
FROM python:3.11                          # [1] tag movida pelo publicador
WORKDIR /app
COPY . .                                  # [4] sem .dockerignore, o .env entra
RUN pip install -r requirements.txt
RUN rm -f .env                            # [4] a camada do COPY conserva
EXPOSE 8000 5678                          # [3] 5678: o depurador, junto
CMD ["uvicorn", "servico:app", "--host", "0.0.0.0", "--port", "8000"]
                                          # [2] sem USER: roda como root
```

O mesmo arquivo da aula de produção, lido por quem ataca.

Quatro defeitos, quatro riscos

	Defeito	Consequência	Risco
[1]	base sem versão fixada	herda a vulnerabilidade de quem a publicou	ASI04
[2]	processo como `root`	execução de código obtida vira **controle do container**	ASI05
[3]	porta além da necessária	depurador e painel alcançáveis pela rede	o caso
[4]	segredo numa camada	`RUN rm` não apaga a camada anterior	ASI03

Correções: base fixada por *digest* · `USER` sem privilégio · só a porta do serviço · `.dockerignore` e segredo por variável da plataforma — e, se vazou, **revogar a chave**.

> O caso de encerramento é o mesmo erro em escala: **padrão inseguro aceito sem leitura**.

`0.0.0.0` no container não é o defeito do caso

```
DENTRO DO CONTAINER   --host 0.0.0.0        necessário: a plataforma
                                            precisa alcançar o processo

NO HOSPEDEIRO         -p 8000:8000          publica em TODAS as
                                            interfaces da máquina
                      -p 127.0.0.1:8000:8000   só o laço local
```

A fronteira está **fora** do container: na porta que a plataforma ou o `docker run -p` publicam.

Fixar versão é decisão de atualização, não congelamento: a base nova passa por build deliberado, varredura e suíte.

Parte 3

MCP, tratado

Três ataques, um mecanismo

A aula de MCP nomeou e parou. O mecanismo comum:

> **A descrição da ferramenta entra no contexto do modelo. Portanto, a descrição é entrada não confiável vinda de terceiro.**

Ataque	Mecanismo
tool poisoning	instruções embutidas na descrição
rug pull	a descrição muda **depois** da aprovação
tool shadowing	um servidor altera o comportamento do agente em relação a **outro**

É **ASI04**. E a escala: ~10 mil servidores públicos, **~1/8** com limiar alto de confiança.

As práticas — gestão de dependência, com um acréscimo

```
FIXAR A VERSÃO da imagem base, do servidor e da ferramenta,
    como se fixa qualquer dependência

REVISAR A DESCRIÇÃO, e não só o código
    <- este é o acréscimo, e é o que quase ninguém faz

VERIFICAR ASSINATURA quando existir
    (no nível de agente, o Agent Card assinado do A2A — em sistemas multiagente)

TRATAR TODA DECLARAÇÃO vinda de fora como ENTRADA,
    e não como configuração
```

A mesma cadeia, uma camada acima: *skills*

As extensões do caso de encerramento eram **skills** — pacotes de instrução e código que o agente instala e executa. A OWASP deu a elas um catálogo próprio, o **Agentic Skills Top 10** (AST01–AST10, 2026):

```
AST01 skill maliciosa                AST06 isolamento fraco
AST02 cadeia de suprimentos          AST07 deriva de atualização
AST03 skill com privilégio demais    AST08 varredura insuficiente
AST04 metadado inseguro              AST09 ausência de governança
AST05 instrução externa              AST10 reuso entre plataformas
      não confiável
```

> **MCP é como o modelo fala com a ferramenta; a skill é o que ela faz.**
>
> AST04 é o *tool poisoning*; AST07 é o *rug pull*. As práticas são as mesmas.

A dependência que o modelo inventa: *slopsquatting*

```
1. o modelo gera código que importa um pacote QUE NÃO EXISTE
2. o nome é plausível, e o mesmo modelo o repete para o mesmo pedido
3. o atacante registra esse nome no índice público
4. quem executa o código sem conferir instala o pacote do atacante
```

Pacotes inexistentes entre os sugeridos: **≥ 5,2%** em modelos comerciais, **21,7%** em modelos abertos (Spracklen et al., 2025). OWASP: LLM03 e LLM09.

> A dependência sugerida pelo modelo é **entrada**: passa pelo crivo de qualquer outra — existência, procedência, versão fixada.

Parte 4

Com que credencial o agente age

O deputado confuso (Hardy, 1988)

O programa que **detém uma autoridade** e a exerce **a pedido de quem não a detém**. É o agente com credencial de serviço (ASI03):

```
CREDENCIAL DE SERVIÇO                    CREDENCIAL DELEGADA
o agente lê todas as despesas            o agente lê as despesas que
de todos os setores                      ESTE usuário pode ler

usuário do setor A pede:                 usuário do setor A pede:
"resuma as despesas do setor B"          "resuma as despesas do setor B"
-> o agente consegue, e resume           -> a ferramenta nega
```

- autorização **na ferramenta, sobre a identidade do usuário** — não no modelo
- credencial **delegada**, com escopo e validade curtos
- ação **do sistema** separada de ação **em nome do usuário**

> À esquerda, a injeção herda o alcance do sistema inteiro; à direita, o de um usuário.

Parte 5

Red team como suíte

A mesma máquina de *evals*, com casos hostis

Estrutura idêntica: conjunto versionado, execução repetida, relatório por fatia.

E o mesmo princípio de origem, com um ajuste:

cada tentativa bem-sucedida vira caso permanente. Exatamente como cada incidente de produção virava.

A suíte hostil **entra no carimbo** — o registro que identifica a configuração e as suítes que a validam — ao lado do *golden dataset* de *evals* — e a aula de sistemas multiagente a roda também sobre a versão multiagente.

Duas métricas, e elas NÃO se misturam

Suíte	Mede	Pergunta
representativa (*evals*)	utilidade	o sistema resolve?
hostil	resistência	o sistema obedece a quem não deveria?

Reportá-las juntas **esconde a troca** que o deck anterior estabeleceu.

As duas colunas

Quadro ilustrativo; os números medidos saem dos scripts `04.x`.

```
                          utilidade    resistência
sem defesa                    87%            0%
+ minimização de contexto     87%           20%
+ confirmação inescapável     85%           60%
+ dual LLM no ramo de RAG     71%           90%
                              ^^^
                              o preço, e ele precisa aparecer
```

O último passo custou **catorze pontos de utilidade** (85% → 71%).

Pode valer a pena — **a decisão exige o número**.

> É o dado que falta em quase toda discussão de segurança de agentes.

Parte 6

Registrar, recuperar, aceitar

O que registrar

O *trace* da aula de arquitetura de agentes é o que torna a investigação possível.

Sem trajetória gravada, **sabe-se que houve dano e não se sabe por onde entrou**.

```
a ORIGEM de cada conteúdo que entrou no contexto
as ferramentas chamadas, com argumentos
as escritas, com identificador
```

E o que fazer quando se descobre

Revogar credencial. E **invalidar a memória contaminada**.

Aqui a aula de memória cobra:

é preciso existir operação de esquecimento seletivo. Se não existir, o sistema **não tem como se recuperar** de um envenenamento — e
cada execução futura **repete o comportamento injetado**.

> É a razão adicional para que o esquecimento exista, e ela não é de privacidade nem de custo: é de **recuperação de incidente**.

Aceitar risco é engenharia

Nem todo risco se mitiga, e a decisão de aceitar um é **legítima quando é declarada**.

```
## RISCO-03 — trifecta letal presente

O agente lê pareceres (privado), recupera documentos (não confiável)
e envia e-mail de notificação (saída).

Por que foi aceito: a notificação é requisito do produto.
O que o limita hoje: destinatário restrito a lista fixa, validada
                     EM CÓDIGO e não configurável por prompt.
O que faria reavaliar: qualquer pedido de destinatário dinâmico.
```

Aceitar risco declarado é engenharia; não declarar é omissão.

O fecho

A superfície

> O sistema construído em **catorze encontros** — que **raciocina** (*prompt engineering*,
> arquitetura de agentes), **recupera** (*embeddings*, RAG), **lembra** (memória), **se conecta** (MCP), **está no ar** (produção) e **é medido** (*evals*, observabilidade) —

é uma superfície de ataque. E agora tem uma suíte que diz **o quanto ele resiste**, ao lado da que diz o quanto acerta.

O que existe, e não foi tratado

O sistema **consome o modelo por API e não o treina**. O que fica de fora pertence a quem treina, hospeda ou opera o modelo em escala pública:

Risco	Fonte	Passa a ser do sistema quando
exemplo adversarial, roubo do modelo, inferência de pertencimento, inversão	MITRE ATLAS	há acesso ao modelo ou aos pesos
envenenamento de treino e de *fine-tuning*, *backdoor*	LLM04	o sistema treina ou ajusta o modelo
índice vetorial compartilhado, inversão do *embedding*	LLM08	o mesmo índice serve a vários clientes
denial of wallet, extração por consulta em massa	LLM10	a API é pública; o teto de gasto só contém

> Não são riscos irrelevantes: é a **condição** em que cada um passa a ser do sistema — o mesmo formato do risco aceito.

O que fica para sistemas multiagente

A lista da nota sobre *tool calling* tem **um item de pé: multiagente**. A aula de sistemas multiagente o risca.

```
ASI07  comunicação insegura entre agentes
       o que atravessa o handoff é ENTRADA NÃO CONFIÁVEL para quem recebe

ASI08  falhas em cascata
       o erro de um agente vira entrada do próximo
```

Dividir o agente em vários abre **fronteiras novas**, e aquela aula as trata como **contrato e como superfície** — com os padrões desta aula e com a suíte hostil rodando sobre as duas versões.